智能体大模型考核项题目：课程助教平台

一、项目定位

为帮助大家在学习数据结构与算法课程时随时提问、练习、查看自己的学习情况，我们要搭建项目数据结构与算法助教平台。做一个 Web 小系统，名字叫数据结构与算法助教平台。学生可以在网页上输入问题、提交代码、查看学习报告。系统要根据学生的问题自动选择用哪种方式回答，包括普通大模型问答、知识库检索问答、调用真实工具、用 CrewAI 生成学习报告。模型自己能回答的可以直接回答，模型自己回答不了的必须通过真实工具完成。

这个平台要帮助学生理解数据结构与算法这门课。老师会提供数据结构与算法的知识库文档，文档放在项目的 knowledge 目录里。学生可以问概念，也可以做题，还可以看自己的掌握情况。

二、实现的功能

第一，知识库问答（必须）。

基于群里提供数据结构与算法知识库文档，内容包括数组、链表、栈、队列等。系统要读取这些文档，切分成小段，做 Embedding，存入向量库。学生提问后，系统先去知识库检索相关段落，再把检索结果交给大模型生成回答，并且返回来源文件名和原文片段。例如学生问“队列的入队和出队时间复杂度是多少”，系统要能检索到队列文档，回答队列是先进先出结构，入队和出队的时间复杂度通常为 O(1)，如果使用循环队列或链表实现可以稳定保持 O(1)。

第二，调用工具（必须）。

系统要用 LangChain Agent，根据学生问题自动决定调用哪个工具。工具必须是真的工具，不是让模型假装调用。以下工具至少要实现三个。

一个是在线代码执行工具。调用 Piston API，地址是 https://emkc.org/api/v2/piston/execute。把学生写的 Python 代码发过去，真实运行，跑测试用例，返回通过情况和报错信息。模型不能自己运行代码，所

以必须用这个外部工具。例如学生提交反转链表的代码，系统要真的运行，看输出对不对。

一个是学生数据查询工具。查询数据库里的做题记录、错题、掌握度。模型不知道学生做过什么，必须查数据库。例如学生问“我链表做了几题”，系统要查数据库后回答。

一个是当前时间工具。返回系统当前时间。模型不知道现在几点，必须调用这个工具。例如学生问“现在几点”，系统要返回真实时间。

一个是维基百科查询工具。调用维基百科摘要 API，地址是 https://zh.wikipedia.org/api/rest_v1/page/summary/关键词。查询算法概念，作为知识库的补充。模型不能实时上网，必须调用这个外部 API。例如学生问“什么是红黑树”，系统可以查维基百科并返回摘要。

工具 api 不一定非得是上面的，根据自身情况，可以选择别的开源搜索 api，但必须真实可用。

第三，学习报告（可选）。

用 CrewAI 做研究员加写手两个 Agent。研究员根据学生真实做题记录、错题、掌握度整理要点。写手根据要点写一份 Markdown 学习报告。报告必须包含真实数据，不能编造。例如学生点“生成本周学习报告”，研究员先查数据库，写手再写报告。报告里要有本周做题数、正确率、各知识点掌握度、薄弱点、下周建议。现场会核对报告里的数字和数据库是否一致。

四、技术要求

开发语言只用 Python。Web 框架用 Django。数据库用 SQLite。大模型 API 用 OpenAI 兼容接口，例如 DeepSeek 或 Qwen。RAG 用 LangChain 加 Chroma 或 FAISS。Agent 用 LangChain。学习报告用 CrewAI。外部工具调用用 requests。环境变量用 python-dotenv。不要引入 React、Vue、Node、Java 等其他技术栈。

附录：在 Django 项目里面调用 CrewAI

第一步，安装 CrewAI，配置环境变量。在.env 里写 OPENAI_API_KEY、OPENAI_BASE_URL、MODEL_NAME。不要提交真实 Key。

第二步，在 Django 项目里新建 services 目录，在 services 里新建 crew_service.py。所有 CrewAI 逻辑都写在这个文件里，不要写在 views.py 里。

第三步,在 crew_service.py 里写函数 generate_learning_report(student_id)。函数内部先查数据库，拿到这个学生的个人学习情况，拼成一段文本。然后定义两个 Agent。研究员 Agent 的 role 是研究员，goal 是整理学习数据，backstory 是擅长分析学习情况。写手 Agent 的 role 是写手，goal 是写报告，backstory 是擅长写技术学习报告。再定义两个 Task。第一个 Task 给研究员，任务是整理学生数据要点。第二个 Task 给写手，任务是根据要点写 Markdown 报告。然后创建 Crew，agents 放两个 Agent，tasks 放两个 Task，process 用 Process.sequential。调用 crew.kickoff()，返回结果。

第四步，在 views.py 里写 POST /api/report/接口。前端传 student_id，视图解析后调用 generate_learning_report，把返回的报告用 JsonResponse 返回。

第五步，注意 CrewAI 运行比较慢，Django 同步请求会等待。简单版可以直接等，前端显示“报告生成中”。更稳的版可以用 threading.Thread 在后台跑，把结果存数据库或全局字典，前端轮询。本次考核允许简单同步版，但要有超时处理。

第六步，不要在每个视图里重复写 Agent 定义。统一封装到 service，视图只负责接收参数和返回 JSON。